Informe de laboratorio 07

Tema: Programación Dinámica: Mochila 0/1 y Mochila Ilimitada

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Laboratorio	Tema	Duración
07	Programación Dinámica: Mochila 0/1 y Mochila Ilimitada	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	29/10/2026	02/11/2026

1. Ejercicios/Problemas propuestos

1.1. Ejercicio 1

Implementa Mochila 0/1 para un conjunto de al menos 8 objetos ingresados por el usuario e imprime tanto el valor máximo como la lista de objetos elegidos.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n, W;
    do {
        cout << "Cantidad de objetos (minimo 8): ";
        cin >> n;
    } while (n < 8);
    do {
        cout << "Capacidad de la mochila (W >= 0): ";
        cin >> W;
    } while (W < 0);
```

```
    vector<int> peso(n), valor(n);
    for (int i = 0; i < n; i++) {
        cout << "Objeto " << (i + 1) << " -> peso y valor: ";
        cin >> peso[i] >> valor[i];
    }
    vector<vector<int> > dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w];
            if (peso[i - 1] <= w) {
                dp[i][w] = max(dp[i][w], valor[i - 1] + dp[i - 1][w - peso[i - 1]]);
            }
        }
    }
    cout << "\nValor maximo (0/1): " << dp[n][W] << endl;
    vector<int> elegidos;
    int w = W;
    for (int i = n; i >= 1; i--) {
        if (dp[i][w] != dp[i - 1][w]) {
            elegidos.push_back(i);
            w -= peso[i - 1];
        }
    }
    reverse(elegidos.begin(), elegidos.end());
    cout << "Objetos elegidos:" << endl;
    int pesoTotal = 0, valorTotal = 0;
    for (size_t k = 0; k < elegidos.size(); k++) {
        int id = elegidos[k];
        cout << "  Objeto " << id << " (peso " << peso[id - 1]
             << ", valor " << valor[id - 1] << ")" << endl;
        pesoTotal += peso[id - 1];
        valorTotal += valor[id - 1];
    }
    cout << "Peso total: " << pesoTotal << " / " << W
         << "   Valor total: " << valorTotal << endl;
    return 0;
}
```

- En este ejercicio se lee la capacidad de la mochila y los objetos con validación de entrada, y la tabla `dp[i][w]` guarda el mayor valor alcanzable usando los primeros i objetos con capacidad w. La transición compara no tomar el objeto contra tomarlo si cabe, y la reconstrucción recorre la tabla desde `dp[n][W]` hacia arriba para devolver la lista de elementos elegidos con su peso y valor totales. Así, el programa no solo obtiene el valor máximo, sino que también ofrece una verificación clara de que la solución cumple la restricción de capacidad.

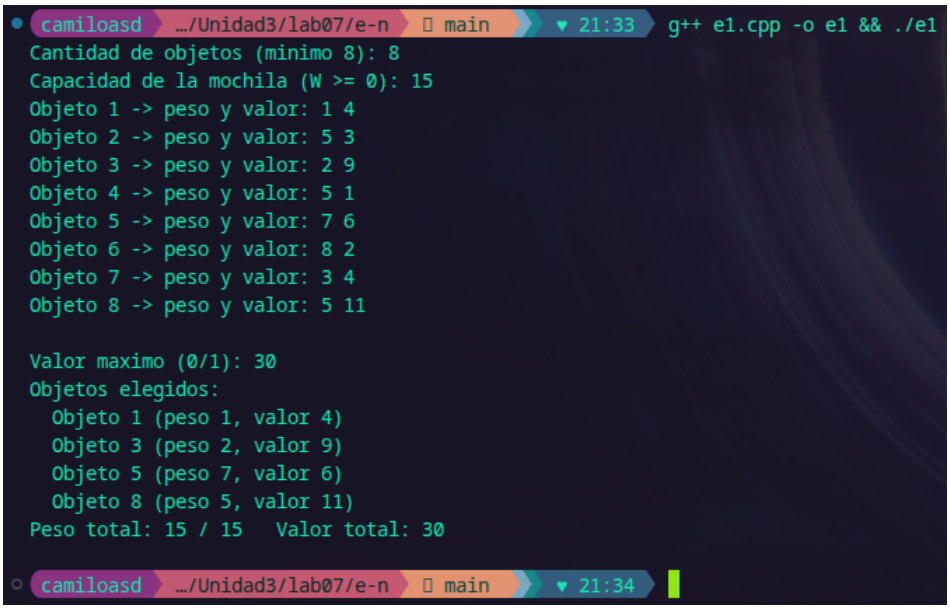


Figura 1: Ejecución del ejercicio 1

1.2. Ejercicio 2

Implementa Mochila Ilimitada para el clásico «problema del cambio de monedas»: dado un conjunto de denominaciones y un monto objetivo, encuentra la combinación que usa la menor cantidad de monedas.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
using namespace std;
int main() {
    int n, M;
    cout << "Cantidad de denominaciones: ";
    cin >> n;
    vector<int> monedas(n);
    cout << "Ingrese las denominaciones (enteros positivos): ";
    for (int i = 0; i < n; i++) cin >> monedas[i];
    cout << "Monto objetivo: ";
    cin >> M;
    const int INF = INT_MAX / 2;
    vector<int> dp(M + 1, INF);
    dp[0] = 0;
    vector<int> eleccion(M + 1, -1);
    for (int m = 1; m <= M; m++) {
        for (int i = 0; i < n; i++) {
            int c = monedas[i];
            if (c <= m && dp[m - c] != INF) {
                if (1 + dp[m - c] < dp[m]) {
                    dp[m] = 1 + dp[m - c];
                    eleccion[m] = c;
```

```
                }
            }
        }
    }
    if (dp[M] == INF) {
        cout << "No es posible formar el monto " << M << " con esas monedas." << endl;
        return 0;
    }
    cout << "\nMinimo de monedas: " << dp[M] << endl;
    cout << "Combinacion: ";
    vector<int> usadas;
    int m = M;
    while (m > 0) {
        usadas.push_back(eleccion[m]);
        m -= eleccion[m];
    }
    sort(usadas.begin(), usadas.end(), greater<int>());
    for (size_t k = 0; k < usadas.size(); k++) {
        cout << usadas[k];
        if (k + 1 < usadas.size()) cout << " + ";
    }
    cout << " = " << M << endl;
    return 0;
}
```

- Este problema se modela como una mochila ilimitada: el monto objetivo es la capacidad, cada denominación es un peso y la decisión consiste en usar una moneda o no, con la diferencia de que la misma moneda puede repetirse tantas veces como sea necesario. El arreglo `dp[m]` guarda el mínimo número de monedas para formar el monto m, y la actualización prueba todas las denominaciones con la recurrencia `1 + dp[m-c]` para conservar la mejor opción. La reconstrucción sigue desde el monto final hasta cero usando la decisión guardada en cada paso, de modo que el programa devuelve la combinación exacta y detecta también cuando un monto es imposible.

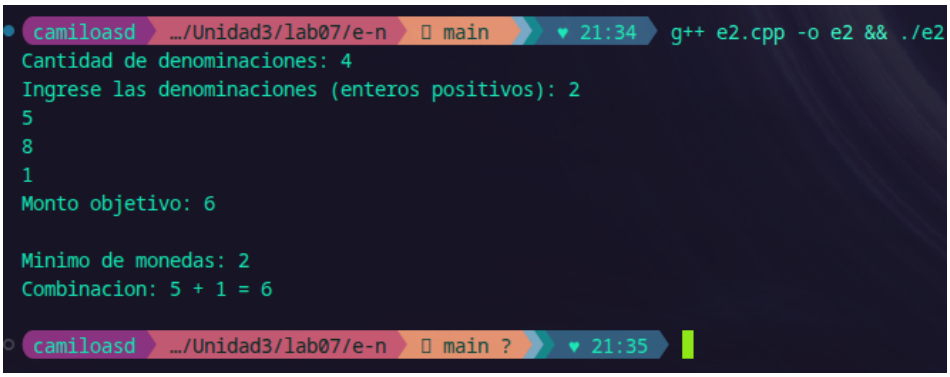


Figura 2: Ejecución del ejercicio 2

1.3. Ejercicio 3

Modifica la solución de Mochila 0/1 para que, además del valor máximo, muestre la tabla ‘dp‘ completa (todas las filas y columnas) y explique cómo se reconstruyó la solución.

```cpp
#include <iostream>
#include <iomanip>
#include <vector>
#include <algorithm>
using namespace std;
void imprimirTabla(const vector<vector<int> >& dp, int n, int W) {
    cout << "\nTabla dp[i][w]  (fila i = \"puedo usar los objetos 1..i\")\n";
    cout << "        ";
    for (int w = 0; w <= W; w++) cout << setw(4) << ("w=" + to_string(w));
    cout << "\n";
    for (int i = 0; i <= n; i++) {
        if (i == 0) cout << "i=0 (-) ";
        else        cout << "i=" << i << " obj" << i << " ";
        for (int w = 0; w <= W; w++) cout << setw(4) << dp[i][w];
        cout << "\n";
    }
}
int main() {
    int n, W;
    cout << "Cantidad de objetos: ";
    cin >> n;
    cout << "Capacidad de la mochila: ";
    cin >> W;
    vector<int> peso(n), valor(n);
    for (int i = 0; i < n; i++) {
        cout << "Objeto " << (i + 1) << " -> peso y valor: ";
        cin >> peso[i] >> valor[i];
    }
    vector<vector<int> > dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w];
            if (peso[i - 1] <= w)
                dp[i][w] = max(dp[i][w],
                               valor[i - 1] + dp[i - 1][w - peso[i - 1]]);
        }
    }
    imprimirTabla(dp, n, W);
    cout << "\nValor maximo = dp[" << n << "][" << W << "] = " << dp[n][W] << "\n";
    cout << "\nReconstruccion (se parte de la esquina dp[" << n << "][" << W << "] y se
    ↪  sube):\n";
    int w = W;
    vector<int> elegidos;
    for (int i = n; i >= 1; i--) {
        cout << "  dp[" << i << "][" << w << "] = " << dp[i][w]
             << "  vs  dp[" << (i - 1) << "][" << w << "] = " << dp[i - 1][w];
        if (dp[i][w] != dp[i - 1][w]) {
            cout << "  -> son DISTINTOS: se llevo el objeto " << i
                 << " (peso " << peso[i - 1] << "), capacidad pasa de "
                 << w << " a " << (w - peso[i - 1]) << "\n";
            elegidos.push_back(i);
            w -= peso[i - 1];
        } else {
            cout << "  -> son IGUALES: NO se llevo el objeto " << i << "\n";
        }
```

```
    }
    reverse(elegidos.begin(), elegidos.end());
    cout << "\nObjetos elegidos: ";
    if (elegidos.empty()) cout << "(ninguno)";
    for (size_t k = 0; k < elegidos.size(); k++) cout << elegidos[k] << " ";
    cout << endl;
    return 0;
}
```

- La diferencia clave de este ejercicio es que la tabla `dp` se imprime completa para que el lector observe la evolución del algoritmo en cada fila y columna. Cada celda compara la opción de no tomar el objeto con la de tomarlo si cabe, y la reconstrucción explica el porqué de cada decisión observando si `dp[i][w]` cambia respecto a `dp[i-1][w]`. Esto permite visualizar, de manera didáctica, cómo se obtiene el valor máximo y qué objetos forman la solución final sin perder la lógica de programación dinámica.

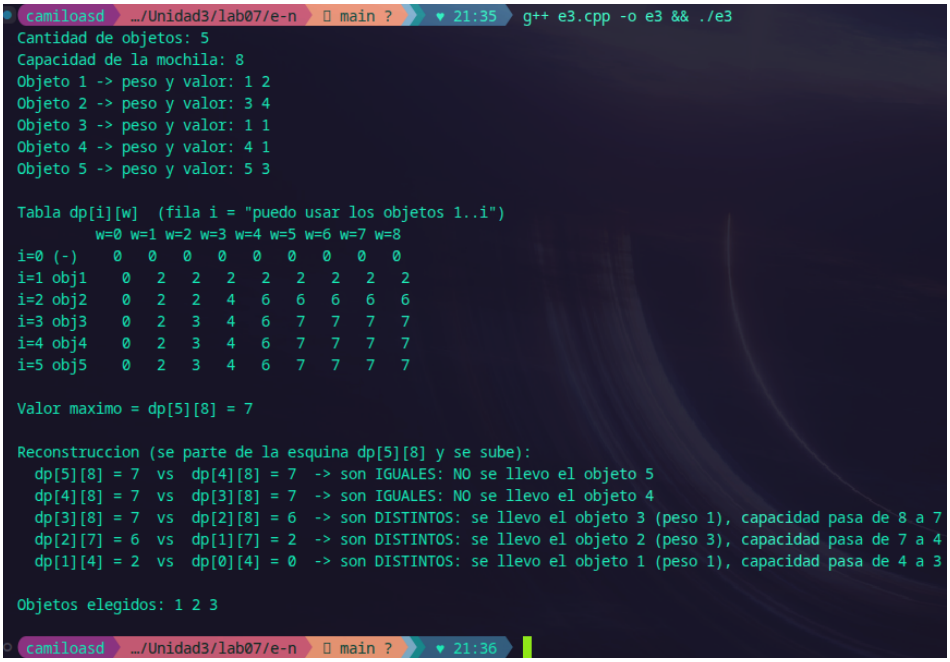


Figura 3: Ejecución del ejercicio 3

1.4. Ejercicio 4

Compara el uso de memoria de Mochila 0/1 (tabla 2D) contra una versión optimizada que use solo un arreglo 1D recorrido de derecha a izquierda. Explica por qué ese recorrido evita reutilizar el mismo objeto más de una vez.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int mochila2D(const vector<int>& peso, const vector<int>& valor, int W) {
```

```
    int n = peso.size();
    vector<vector<int> > dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w];
            if (peso[i - 1] <= w)
                dp[i][w] = max(dp[i][w], valor[i - 1] + dp[i - 1][w - peso[i - 1]]);
        }
    return dp[n][W];
}
int mochila1D(const vector<int>& peso, const vector<int>& valor, int W) {
    int n = peso.size();
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        for (int w = W; w >= peso[i]; w--)
            dp[w] = max(dp[w], valor[i] + dp[w - peso[i]]);
    }
    return dp[W];
}
int mochila1D_alReves(const vector<int>& peso, const vector<int>& valor, int W) {
    int n = peso.size();
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int w = peso[i]; w <= W; w++)
            dp[w] = max(dp[w], valor[i] + dp[w - peso[i]]);
    return dp[W];
}
int main() {
    int n, W;
    cout << "Cantidad de objetos: ";
    cin >> n;
    cout << "Capacidad de la mochila: ";
    cin >> W;
    vector<int> peso(n), valor(n);
    for (int i = 0; i < n; i++) {
        cout << "Objeto " << (i + 1) << " -> peso y valor: ";
        cin >> peso[i] >> valor[i];
    }
    int r2 = mochila2D(peso, valor, W);
    int r1 = mochila1D(peso, valor, W);
    int rMal = mochila1D_alReves(peso, valor, W);
    cout << "\nValor maximo con tabla 2D          : " << r2 << endl;
    cout << "Valor maximo con arreglo 1D (der->izq): " << r1 << endl;
    cout << "Valor con arreglo 1D (izq->der, MAL)   : " << rMal
         << (rMal != r1 ? "   <- distinto: reutiliza objetos" : "   (coincide en este
         ↪  caso)") << endl;
    cout << (r1 == r2 ? "Las versiones correctas coinciden.\n" : "ERROR: no coinciden.\n");
    long long mem2D = (long long)(n + 1) * (W + 1) * sizeof(int);
    long long mem1D = (long long)(W + 1) * sizeof(int);
    cout << "\nMemoria tabla 2D: (n+1)*(W+1) = " << (n + 1) << "*" << (W + 1)
         << " enteros = " << mem2D << " bytes" << endl;
    cout << "Memoria arreglo 1D: (W+1) = " << (W + 1)
         << " enteros = " << mem1D << " bytes" << endl;
    cout << "Ahorro: " << (n + 1) << " veces menos memoria (O(n*W) -> O(W))." << endl;
    cout << "Nota: con 1D se pierde la tabla completa, por lo que ya no se puede\n"
            "reconstruir facilmente que objetos se eligieron.\n";
```

```
    return 0;
}
```

- Aquí se compara la versión 2D, que guarda toda la tabla y permite reconstruir la solución, con la versión optimizada en 1D que usa un solo arreglo para reducir la memoria a $O(W)$. La clave está en recorrer la capacidad de derecha a izquierda, porque así la posición `dp[w - peso]` aún conserva el valor de la fila anterior y no se reutiliza el mismo objeto en la misma pasada. Si se recorre de izquierda a derecha, la celda ya queda actualizada con el mismo objeto y la solución se comporta como una mochila ilimitada; el programa demuestra exactamente esta diferencia y la comparación de memoria entre ambas versiones.

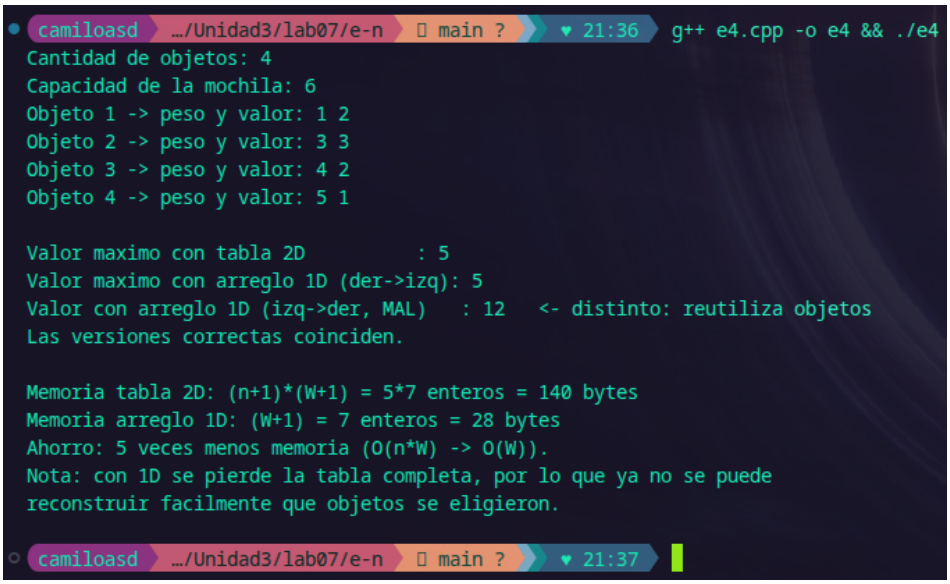


Figura 4: Ejecución del ejercicio 4

1.5. Ejercicio 5

Aplica Mochila 0/1 a un caso real: dado un presupuesto fijo y una lista de posibles compras (con costo y beneficio estimado), determina la combinación que maximiza el beneficio sin exceder el presupuesto.

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;
int main() {
    int n, presupuesto;
    cout << "Presupuesto disponible (entero, ej. soles): ";
    cin >> presupuesto;
    cout << "Cantidad de compras posibles: ";
    cin >> n;
    vector<string> nombre(n);
```

```
    vector<int> costo(n), beneficio(n);
    cin.ignore();
    for (int i = 0; i < n; i++) {
        cout << "Compra " << (i + 1) << " - nombre: ";
        getline(cin, nombre[i]);
        cout << "           costo y beneficio estimado (ej. 1200 8): ";
        cin >> costo[i] >> beneficio[i];
        cin.ignore();
    }
    vector<vector<int> > dp(n + 1, vector<int>(presupuesto + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int p = 0; p <= presupuesto; p++) {
            dp[i][p] = dp[i - 1][p];
            if (costo[i - 1] <= p)
                dp[i][p] = max(dp[i][p],
                               beneficio[i - 1] + dp[i - 1][p - costo[i - 1]]);
        }
    }
    vector<int> elegidas;
    int p = presupuesto;
    for (int i = n; i >= 1; i--) {
        if (dp[i][p] != dp[i - 1][p]) {
            elegidas.push_back(i - 1);
            p -= costo[i - 1];
        }
    }
    reverse(elegidas.begin(), elegidas.end());
    cout << "\n=== Plan de compras optimo ===\n";
    int gasto = 0;
    for (size_t k = 0; k < elegidas.size(); k++) {
        int j = elegidas[k];
        cout << " - " << left << setw(22) << nombre[j]
             << " costo " << setw(6) << costo[j]
             << " beneficio " << beneficio[j] << "\n";
        gasto += costo[j];
    }
    if (elegidas.empty()) cout << " (ninguna compra cabe en el presupuesto)\n";
    cout << "Gasto total: " << gasto << " de " << presupuesto
         << "   Sobrante: " << (presupuesto - gasto) << "\n";
    cout << "Beneficio total maximo: " << dp[n][presupuesto] << endl;
    return 0;
}
```

- En este caso cada compra se convierte en un objeto con peso igual al costo y valor igual al beneficio estimado, mientras que el presupuesto representa la capacidad de la mochila. La tabla `dp` calcula el mejor beneficio posible para cada nivel de gasto disponible y, luego, la reconstrucción vuelve hacia atrás para elegir exactamente las compras que forman el plan óptimo. La salida final incluye el nombre, costo, beneficio, gasto total y sobrante, por lo que la solución no solo maximiza el valor, sino que además queda validada frente al presupuesto real.

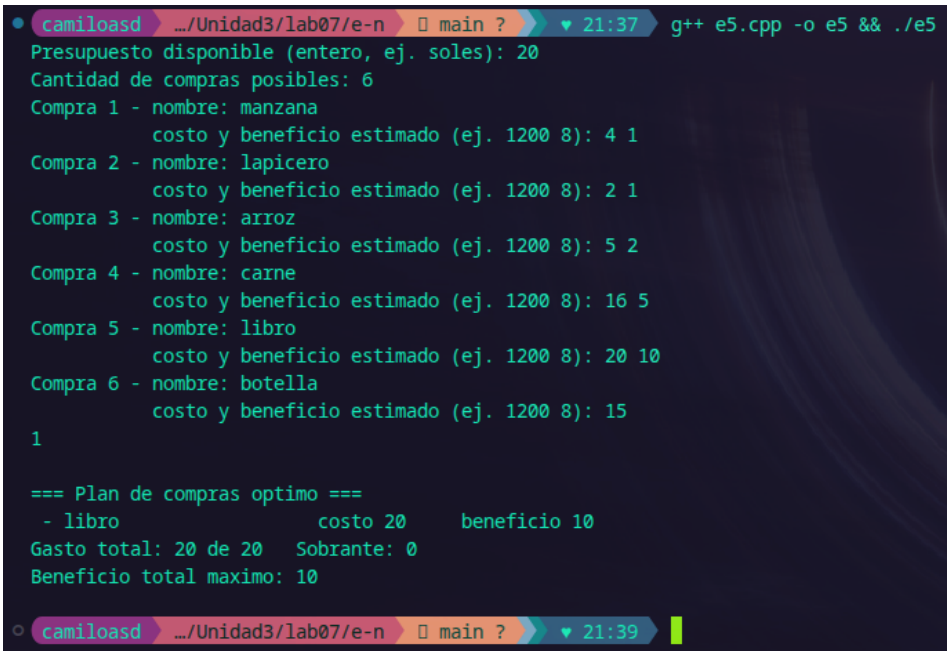


Figura 5: Ejecución del ejercicio 5

2. Cuestionario

1. ¿Por qué Mochila 0/1 necesita una tabla bidimensional, mientras que Mochila Ilimitada se resuelve con un arreglo unidimensional?
   Porque en Mochila 0/1 cada objeto puede elegirse o no elegirlo una sola vez, por lo que el estado debe recordar qué objetos ya fueron considerados; por eso se requiere la dimensión `i` para distinguir entre los primeros i objetos y la dimensión `w` para la capacidad restante. En la versión ilimitada, la misma moneda u objeto puede reutilizarse, así que solo basta recordar el mejor valor para cada capacidad, lo que se representa con un solo arreglo `dp[w]`.

2. En Mochila 0/1, ¿qué representa exactamente el valor `dp[i][w]`?
   Representa el máximo beneficio (o valor) que se puede obtener usando solo los primeros i objetos y una capacidad total de w. Es decir, es el mejor resultado posible para ese subproblema, sin exceder la capacidad disponible.

3. ¿Por qué en el ejemplo resuelto Mochila Ilimitada obtiene un valor mayor que Mochila 0/1 con la misma capacidad?
   Porque en Mochila Ilimitada un mismo elemento puede aparecer varias veces, mientras que en Mochila 0/1 cada objeto se toma a lo más una vez. Por ello, la versión ilimitada puede repetir una denominación o una compra, lo que permite acumular más valor o formar montos más grandes con la misma capacidad.

4. ¿Cómo se podría adaptar Mochila 0/1 para permitir llevar cada objeto hasta un número limitado de veces (por ejemplo, máximo 3 unidades de cada uno)?
   Se puede extender el estado con una tercera dimensión que indique cuántas copias del objeto actual ya se han usado. En la práctica, para cada objeto i y cada capacidad w, se prueba si se toman 0, 1, 2 o 3 unidades del mismo elemento, y se actualiza la tabla usando el valor correspondiente. También se puede convertir este problema en un caso de mochila con multiplicidad limitada, donde el objeto tiene varias versiones de copia.

5. ¿Qué papel cumple el paso B (casos base) de SRTBOT en ambas variantes de la mochila?
   El caso base define los estados iniciales del problema: cuando no hay objetos disponibles o la capacidad es cero, el valor máximo es cero. Esto evita decisiones ambiguas y garantiza que la recurrencia comience desde una base correcta; en la mochila ilimitada, además, se inicializa `dp[0] = 0` y los demás estados con infinito si son imposibles.

3. Conclusiones

La programación dinámica fue la herramienta clave para resolver los problemas planteados porque permite dividir un problema complejo en subproblemas más pequeños y reutilizar resultados ya calculados. En la mochila 0/1 y en la mochila ilimitada, la estructura recursiva permite decidir si conviene incluir o excluir un elemento sin perder la optimización global, y la tabla `dp` refleja exactamente ese proceso.

Durante el laboratorio se observó que la diferencia entre ambas variantes no está solo en la fórmula, sino en la interpretación del estado: en mochila 0/1 cada objeto se considera una sola vez, mientras que en la versión ilimitada se permite reutilizarlo con el mismo costo de una nueva elección. Esta diferencia cambia completamente la forma de recorrer la tabla y la complejidad práctica de la solución.

Además, los ejercicios demostraron que la programación dinámica no solo resuelve el valor óptimo, sino que también permite reconstruir la solución final, algo esencial cuando se desea mostrar qué objetos o compras forman la mejor elección. En conjunto, la práctica mostró que esta técnica es útil no solo en teoría, sino también en aplicaciones reales como la planificación de compras, el cambio de monedas y la optimización de recursos.